C−− (C Minus Minus): Design and Implementation of a Memory-Safe Imperative Language, Compiler, and Web-Based Compiler Studio

Project Report

Supervised by Prof. Hariharan R L, Center for AI Research, School of Computer Science and Engineering (SCOPE)

Ronit Mexson
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BAI0036

Mit Kumar Raju Panchal
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BAI0279

Garv Gupta
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BDS0169

***Abstract*—This report presents the design, implementation and evaluation of C−− (C Minus Minus), a statically typed, C-like imperative programming language built on the principle of *safety by removal*. Rather than adding garbage collection, ownership tracking or runtime sanitizers to make dangerous features safe, C−− removes the features themselves: there are no pointers, no null, no uninitialized variables, no implicit conversions and no undefined behaviour. Bindings are immutable by default, collections have deep-copy value semantics, and every remaining fault is a deterministic, positioned runtime trap. The project delivers a complete toolchain written in pure Python with no external dependencies: a lexer, a recursive-descent parser, a semantic analyzer, a reference interpreter, an ISO C99 transpiler, a diagnostics system with 60 documented error codes, a command-line tool, a VS Code extension, and a server-rendered web compiler studio with live diagnostics. The report describes each component, the 445-test quality-assurance process run on three operating systems, the deployment, and an evaluation of safety and performance, including a case study that uses C−− to model AVX-512 vector predication.**

***Index Terms*—programming language design, memory safety, compiler construction, static analysis, interpreters, source-to-source translation, compiler diagnostics, web-based development environments**

I. Introduction

C and C++ remain the foundation of operating systems, embedded software and performance-critical applications, but their raw pointers, manual memory management and long list of undefined behaviours make them the leading source of severe security defects. Microsoft reports that about 70% of the vulnerabilities it assigns a CVE each year are memory-safety issues [1], the Chromium project reports a similar share among its serious security bugs [2], and government agencies now ask software vendors to publish memory-safety roadmaps [3].

Memory-safe alternatives exist, but each adds complexity: garbage-collected languages add a runtime, and Rust adds ownership, borrowing and lifetimes that beginners find hard to learn [4]. This project asks a different question: *what is left of C's simple, explicit style if every construct capable of undefined behaviour is removed instead of guarded?* The answer is C−−, a language small enough to learn quickly and to implement completely within a student project, yet strict enough that the most common classes of memory-corruption and arithmetic defects either cannot be written or stop the program with a clear message.

A. Objectives

1) Design a statically typed imperative language whose grammar and type system exclude null, pointers, uninitialized reads, implicit narrowing and undefined behaviour.
2) Specify the language normatively, down to exact error messages, number formats and source positions, so that every behaviour is testable.
3) Implement a complete, dependency-free toolchain: front end, reference interpreter and a C99 back end.
4) Make diagnostics understandable to beginners, with explanations, fixes and spelling suggestions.
5) Provide tooling for adoption: a CLI, an editor extension and a browser-based compiler studio.
6) Validate the implementation with automated tests on Windows, macOS and Linux, and evaluate its safety and performance.

B. Scope and organization

Version 0.2.0 of C−− is covered. Section II reviews related work and Section III states the requirements. Section IV gives the system architecture, Section V the language, and Section VI the compiler implementation. Sections VII–IX describe the diagnostics, the developer tooling and the web studio. Section X covers testing and deployment, Section XI the evaluation, and Section XII limitations and future work.

II. Literature Survey

Safe dialects of C. Cyclone [5] keeps C's syntax but restricts pointer arithmetic and adds region-based memory management with several pointer kinds. CCured [6] infers

pointer kinds for legacy C and inserts run-time checks. Checked C [7] adds checked pointer and array types that can be adopted incrementally. All three keep pointers and must therefore track them; C−− removes them.

Ownership-based safety. Rust [4] guarantees memory safety without a garbage collector through ownership, borrowing and lifetimes. C−− achieves alias freedom far more simply, if less efficiently, by giving collections value semantics.

Dynamic detection. AddressSanitizer [8] instruments C and C++ programs to detect out-of-bounds and use-after-free accesses during testing, but offers no guarantee in production.

Language subsets. MISRA C [9] restricts C through coding rules checked by external tools, and SPARK [10] restricts Ada so that programs can be formally verified. C−− follows the same subsetting philosophy but makes the restrictions part of the language itself.

Compiler diagnostics. Studies of novice programmers find terse compiler messages a major obstacle [11]; Elm popularized messages written for humans [12]. C−− applies this style to every diagnostic and adds spelling suggestions based on edit distance [13], [14].

Compiler construction. The implementation follows classical techniques: hand-written scanning, recursive-descent parsing by precedence levels, and syntax-directed semantic checking [15]. The name C−− is unrelated to the intermediate language of the same name used inside the Glasgow Haskell Compiler [16].

III. Requirements

A. Functional requirements

- **FR1:** Parse and statically check programs with functions, four primitive types, four generic collections, structured control flow and `switch`.
- **FR2:** Execute checked programs with fully defined semantics, trapping every fault with a positioned runtime error.
- **FR3:** Translate programs to portable ISO C99 that prints exactly what the interpreter prints.
- **FR4:** Report every error with a code, a position, an explanation and a suggested fix.
- **FR5:** Provide CLI commands to run, check, tokenize, print the AST and transpile.
- **FR6:** Provide a web interface for writing, running and inspecting programs without installation.

B. Non-functional requirements

- **Portability:** identical behaviour and byte-identical output on Windows, macOS and Linux with Python 3.10 or newer.
- **Zero dependencies:** only the Python standard library at run time.
- **Determinism:** no undefined or implementation-defined behaviour, including float printing and integer division.
- **Robustness:** bounded parser nesting (200), call depth (1000) and optional execution steps, so untrusted programs cannot exhaust the host.

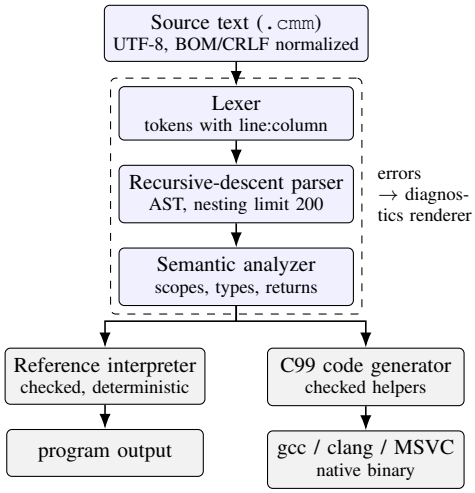


Fig. 1. Compiler pipeline. Each stage reports positioned errors to a shared diagnostics renderer.

TABLE I
Repository structure

Path	Contents
`cmm/`	Language implementation: 20 modules, 6 304 lines
`tests/`	445 tests in 11 files and a corpus of 61 error programs
`web/`	Web studio: HTTP server, page renderer, client application
`docs/`	Normative specification (v0.1 and v0.2), error catalogue, changelog
`examples/`	Example programs with expected output
`extensions/`	VS Code extension: grammar, language configuration, icons
`case_studies/`	AVX-512 SIMD predication case study
`scripts/`	Executable builder, documentation and test generators
`paper/`	This report, the IEEE paper and their benchmarks

- **Specification first:** a normative specification is the single source of truth; behaviour not specified is not implemented.

IV. System Architecture

Fig. 1 shows the compiler pipeline and Table I the structure of the repository. The `cmm` Python package contains the whole language implementation; the CLI, the web server and the tests are thin clients of it.

V. Language Specification

C−− is specified normatively in two documents (v0.1.0 and the v0.2.0 addendum) that fix every token, message, position and output format. A *rule against assumptions* requires the implementation to stop and ask rather than guess wherever the specification is silent.

A. Design principles

(i) Remove rather than guard: a feature whose misuse causes a vulnerability class is omitted unless it can be given

TABLE II
TYPES OF C−− V0.2.0

Type	Semantics
`int`	64-bit signed; overflow traps (R001)
`float`	IEEE 754 binary64; never Inf or NaN (R002, R004)
`bool`	`true`/`false`; never converted to or from numbers
`string`	immutable Unicode text; indexing by code point, `.len()`
`array<T>`	fixed length, bounds-checked indexing (R006)
`list<T>`	growable; `push`, `pop`, `insert`, `remove`, `contains`
`stack<T>`	LIFO; `push`, `pop`, `peek`; empty pop traps (R012)
`queue<T>`	FIFO; `enqueue`, `dequeue`, `peek`

fully defined semantics. *(ii) No undefined behaviour:* every program has a specified result or stops with a specified, positioned error. *(iii) Explicitness:* no implicit narrowing, no truthiness, no context-dependent operators.

B. Lexical structure

Source files are strict UTF-8; a byte-order mark is removed and CRLF line endings are normalized before lexing, and columns count Unicode code points. Integer literals are decimal and may not have leading zeros; float literals need digits on both sides of the point. Strings may use single, double or triple quotes with six escape sequences. The lexer rejects `++`, `--`, single `&` and single `|` with a hint (L009–L012). There are 25 active keywords and 14 reserved words kept for future versions.

C. Types

C−− has four primitive types and four generic collections (Table II). Integers are 64-bit and every operation is range-checked; division truncates toward zero. Floats follow IEEE 754 binary64 [17], but infinities and NaN can never arise: overflow or division by zero is an error. The only implicit conversion widens an `int` operand to `float` in mixed arithmetic or comparison; assignment never converts.

D. Grammar

Listing 1 gives the core of the v0.2.0 grammar in EBNF. Expressions follow the precedence of Table III; all binary operators are left-associative.

E. Static semantics

Every variable is declared with `let` (immutable) or `mut` (mutable) and must be initialized; its type is inferred. Conditions must be `bool`. Assignment is a statement, so `if (x = 1)` is a syntax error. Braces are mandatory. Function signatures are collected before bodies are checked, so functions may be defined in any order. A conservative *definitely-returns* rule rejects a typed function with any path that does not return (E013). A `switch` accepts `int` or `string` literals, has no fall-through and rejects duplicate cases.

TABLE III
OPERATOR PRECEDENCE (HIGHEST FIRST)

Level	Operators	Meaning
1	`a[i]`, `f(x)`, `v.m(x)`	postfix: index, call, method
2	`!`, `-`	prefix: logical not, negation
3	`*`, `/`, `%`	multiplicative
4	`+`, `-`	additive
5	`<`, `<=`, `>`, `>=`	comparison
6	`==`, `!=`	equality
7	`&&`	logical and (short-circuit)
8	`||`	logical or (short-circuit)

TABLE IV
SIZE OF THE LANGUAGE IMPLEMENTATION

Component (modules)	Lines
Lexer and token definitions (`lexer`, `tokens`)	585
Parser and AST (`parser`, `ast_nodes`)	1 343
Semantic analyzer and type rules (`analyzer`, `types`)	778
Interpreter and runtime (`interpreter`, `runtime_*`, `builtins`)	1 117
C99 code generator (`codegen_c`)	859
Diagnostics (`errors`, `explanations`, `suggest`, `diagnostic_render`)	867
CLI and printers (`cli`, `astprint`, `formatting`, others)	755
Total	**6 304**

F. Dynamic semantics

Assigning, passing or returning a collection copies it deeply, so two names never share data and a callee cannot modify its caller's variables (Listing 2). Immutability is deep: a `let` collection allows neither element assignment nor mutating methods. Each call gets a fresh environment containing only its parameters; there are no globals. Recursion is limited to 1000 nested calls (R007).

VI. COMPILER IMPLEMENTATION

The implementation is about 6 300 lines of Python using only the standard library (Table IV).

A. Lexer

The hand-written scanner reads the normalized text once, using longest match for operators, and produces tokens carrying their type, lexeme, value and 1-based line and column. Number literals are validated and range-checked during scanning (L005–L008). The lexer stops at the first error.

B. Parser

The parser is a stratified recursive-descent parser [15] with one procedure per precedence level of Table III, which yields C's usual precedence and left associativity without a separate precedence table. A statement that begins with an identifier is classified by its next token: call, method call or assignment. Every nested construct increments a depth counter, and nesting beyond 200 levels raises S004 instead of overflowing the Python stack.

```
program    = "module" IDENT ";" function { function } EOF ;
function   = "fn" ( IDENT | "main" ) "(" [ param { "," param } ] ")"
             [ "->" type ] block ;
param      = IDENT ":" type ;
type       = "int" | "float" | "bool" | "string"
           | ( "array" | "list" | "stack" | "queue" ) "<" elemType ">" ;
block      = "{" { statement } "}" ;
statement  = varDecl | assignStmt | callStmt | printStmt | returnStmt
           | ifStmt | whileStmt | doWhileStmt | forStmt | switchStmt | block ;
varDecl    = ( "let" | "mut" ) IDENT "=" expr ";" ;
assignment = IDENT [ "[" expr "]" ] ( "=" | "+=" | "-=" | "*=" | "/=" | "%=" ) expr ;
ifStmt     = "if" "(" expr ")" block [ "else" ( ifStmt | block ) ] ;
forStmt    = "for" "(" forInit [ expr ] ";" [ assignment ] ")" block ;
switchStmt = "switch" "(" expr ")" "{" caseClause { caseClause }
             [ "default" ":" block ] "}" ;
```

Listing 1. Core grammar of C−− v0.2.0 (EBNF, abridged).

```
module main;

fn mean(xs: list<float>) -> float {
    mut total = 0.0;
    for (mut i = 0; i < xs.len(); i = i + 1) {
        total += xs[i];
    }
    return total / xs.len();
}

fn main() {
    mut samples = list<float>();
    samples.push(2.5);
    samples.push(4.0);
    let snapshot = samples;    // a deep copy
    samples.push(9.5);
    print(mean(samples));      // 5.333333333333333
    print(snapshot.len());     // 2
}
```

Listing 2. Typed functions, a growable list and deep-copy value semantics.

C. Semantic analyzer

The analyzer makes two passes. The first records all function signatures and checks them against the built-in function table; the second walks each body with a stack of lexical scopes, inferring and checking types, enforcing mutability, validating calls and collection methods against a declarative method table, checking `switch` cases and applying the definitely-returns rule. All semantic errors are collected and reported in source order.

D. Reference interpreter

The interpreter is a tree-walking evaluator over the verified AST. It defines the language's semantics:

- every `int` result is checked against $[-2^{63}, 2^{63} - 1]$ (R001), every float result for overflow (R004), and every division and remainder for a zero divisor (R002, R003);
- collections are copied on assignment, call and return;
- it runs in a thread with a 256 MB stack, so the 1000-call recursion limit behaves identically on every platform;
- an optional step budget stops runaway loops (R005); the web server always sets one;
- floats are printed with a shortest round-trip algorithm specified in the language definition.

TABLE V
DIAGNOSTIC CODES BY PHASE

Prefix	Phase	Codes	Examples
L	Lexical	12	++ not supported, unterminated string
S	Syntax	3	expected token, reserved word, nesting
E	Semantic	30	undeclared name, `let` reassigned, missing return
R	Runtime	14	overflow, division by zero, index range
T	Transpile	1	feature not supported by the C back end
	Total	**60**	

E. C99 code generator

The transpiler lowers the v0.1 subset plus user-defined functions to ISO C99 [18] using only standard headers. Arithmetic becomes calls to small `static` helpers that perform the interpreter's overflow and division checks, and a custom float formatter reproduces the interpreter's output exactly. Collections, indexing and `switch` are not yet supported; such programs are rejected with T001.

VII. Diagnostics

The 60 diagnostic codes (Table V) are defined in one catalogue, from which the reference documentation and the golden tests are generated. Every diagnostic has a positioned headline, a source excerpt with a caret under the offending text, a plain-language explanation and a concrete fix, often with corrected code. For undeclared names the analyzer suggests the closest declared identifier by Damerau–Levenshtein distance [13], [14], allowing one edit for names shorter than five characters and two otherwise (Listing 3). The `--no-help` flag restores a terse format for tools.

VIII. Developer Tooling

A. Command-line interface

The `cmm` command (Table VI) is installed by `pip` or run as `python3 -m cmm`. Output is always UTF-8 with `\n` line endings, and exit codes distinguish static errors (1), runtime

```
typo.cmm:5:19: semantic error: undeclared variable
      'maximum_scor'
  |
5 |     mut current = maximum_scor + 10;
  |                   ^~~~~~~~~~~~
what went wrong:
  'maximum_scor' has not been declared in this scope.
how to fix:
  Declare 'maximum_scor' before using it, with 'let
      maximum_scor = ...;' or
  'mut maximum_scor = ...;'.
      let maximum_scor = 0;
  did you mean 'maximum_score'?
```

Listing 3. Diagnostic for a misspelled identifier, as printed by `cmm run`.

TABLE VI
CLI COMMANDS

Command	Action
`cmm run f [--max-steps N]`	check and execute
`cmm check f`	static checks only
`cmm tokens f`	print the token stream with positions
`cmm ast f`	print the abstract syntax tree
`cmm transpile f -o out.c`	generate ISO C99
`cmm --version`	print `cmm 0.2.0`

traps (2), I/O errors (3), usage errors (4) and internal errors (70). A PyInstaller script builds a single-file executable.

B. Editor support

A VS Code extension contributes the C−− language definition, a TextMate grammar for syntax highlighting, bracket and comment configuration and a file-icon theme. Platform scripts register the `.cmm` file type and icon on Windows and macOS.

IX. Web Compiler Studio

The web studio lets users write and run C−− programs in a browser without installing anything. It is written with the standard library alone (about 4 700 lines of Python, JavaScript, HTML and CSS) and is deployed publicly. Fig. 2 shows its architecture.

A. Server and API

The server renders six pages (overview, playground, architecture, pipeline, reference, about) on the server, so each has its own URL and works without JavaScript. The JSON API (Table VII) wraps the compiler pipeline. Untrusted programs run with a 100 000-step budget and an empty standard input, request bodies are limited to 1 MB, and each request is served on its own thread so that a long-running program cannot block others.

B. Client features

The editor provides syntax highlighting, automatic indentation, bracket pairing, comment toggling and diagnostics that update as the user types: errors are marked beside the line numbers and listed in a Problems panel, and every error location links back to the source. Inspectors show the console output, the generated C99, the AST and a clickable token table, and a stage trace shows which phase passed or failed. A command palette, keyboard shortcuts, saved drafts and shareable links complete the interface.

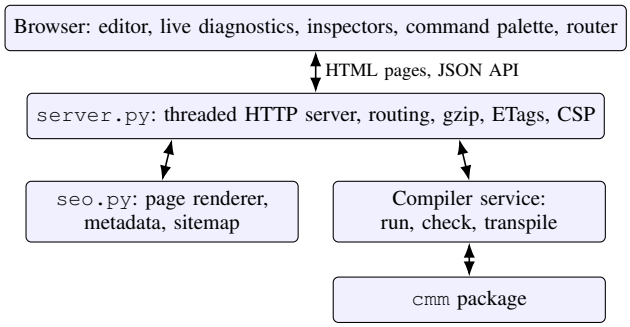


Fig. 2. Architecture of the web compiler studio.

TABLE VII
WEB STUDIO API

Method	Endpoint	Result
POST	`/api/run`	output, or diagnostics with failing stage
POST	`/api/check`	static checks only, no execution
POST	`/api/transpile`	generated C99 source
POST	`/api/ast`	formatted syntax tree
POST	`/api/tokens`	token stream with positions
GET	`/api/presets`	example programs

C. Security, performance and discoverability

Static files are served from an allow-list with path-traversal protection. Every page carries a strict Content-Security-Policy with a hashed inline script, together with `nosniff` and referrer-policy headers. Responses are compressed, and versioned assets are cached immutably. Each page has unique metadata, Open Graph tags and schema.org structured data, and the server publishes a sitemap and `robots.txt`. Lighthouse audits of all six pages scored 97–100 in performance, accessibility, best practices and SEO.

X. Quality Assurance and Deployment

A. Test suite

Table VIII lists the 445 automated tests. The error-help tests run each of the 61 programs in the error corpus and compare the complete diagnostic with a golden file character for character. The example tests run every example through the CLI, and the code-generation tests compile the transpiled C99 with `-Werror` and require output identical to the interpreter's.

B. Continuous integration

GitHub Actions runs the full suite on a matrix of three operating systems (Ubuntu, Windows, macOS) and three Python versions (3.10, 3.11, 3.12), nine jobs in total, followed by a CLI smoke test. On our machine 444 of the 445 tests pass. The single failure is a portability defect in the C back end: for a literal equal to $2^{63} - 1$ the generated code prints an `LL` constant with the `PRId64` format, which GCC rejects under `-Werror` where `int64_t` is `long`. Casting the constant to `int64_t` fixes it.

TABLE VIII
AUTOMATED TESTS

File	Covers	Tests
test_lexer	tokens, literals, lexical errors	40
test_parser	grammar, AST, syntax errors	46
test_analyzer	scopes, types, returns	42
test_interpreter	semantics, runtime traps	51
test_v02	functions, collections, switch	49
test_formatting	value and float printing	26
test_codegen_c	C99 back end against interpreter	16
test_cli	commands, exit codes, I/O	39
test_error_help	61 golden diagnostics and helpers	78
test_examples	example programs end to end	30
test_web_seo	web server, pages, API	28
Total		**445**

C. Deployment

The studio is packaged three ways: a `Dockerfile` based on `python:3.12-slim`, a `Procfile` for platforms such as Railway, and a `render.yaml` blueprint. All three install the package and start `web/server.py`, which reads its port and public URL from environment variables. Locally the studio starts with `python3 web/server.py`.

XI. Results and Evaluation

A. Safety guarantees

Table IX maps common weakness classes to the C−− rule that removes them and the stage at which a violating program is stopped. Classes in the upper part cannot be written at all; those in the lower part are expressible but become deterministic, positioned runtime traps. Every code in the table is exercised by a program in the test corpus.

B. Performance

Table X reports two experiments on four benchmarks: recursion, integer loops with division, data-dependent control flow and floating point. The first compares the interpreter with the same program transpiled to C99 and compiled with `gcc -O2`; all outputs matched between the two back ends. The second measures the cost of the safety checks by comparing the transpiled C99 with hand-written, unchecked C. Times are the best of several runs on Intel Xeon Processor @ 2.10GHz, with GCC 13.3.0 and Python 3.13.16. Native times include process start-up, which dominates in Experiment 1, so its speedups are lower bounds.

The transpiled C99 is 225–878 × faster than the interpreter, whose purpose is to define the semantics rather than to be fast. The checked C99 is 29–217% slower than unchecked C; this is the price of the guarantees. The cost is lowest when work is dominated by control flow (Collatz) and highest for recursive Fibonacci and the floating-point series, whose every division and result is checked.

C. Case study: modelling SIMD predication

To test C−− as a modelling tool, we used it to study a classic vectorization problem: a loop computing `B[i] = A[i] > 0 ? sqrt(A[i]) : 0`. A C−− program modelled 512-bit AVX-512 registers as 16-lane `array<float>` values and mask registers as `array<bool>` values. It executed the loop four ways (scalar with a branch, AVX-512 masked, AVX-512 using `max`, AVX2 blend) and verified lane by lane that all four agree. It also simulated a 2-bit and a two-level branch predictor and estimated instruction counts.

A hardware benchmark on an Intel Xeon with AVX-512 confirmed the model: predication removed every data-dependent branch, cut dynamic instructions per element by about 18–21× (8 to 0.375–0.44) and branches by 32×, and made the loop about 19× faster on random data while its run time became independent of the data. One finding was specific to C−−: removing the tail mask from a modelled load made the program stop with R006 instead of reading past the array, the very fault that hardware masked loads exist to prevent [19].

XII. Limitations and Future Work

- **Expressiveness:** no structs, enumerations, maps, nested collections, first-class functions or string concatenation. User-defined record types are the next planned feature.
- **Copy cost:** deep-copying collections is expensive for large data; copy-on-write would keep the semantics at lower cost.
- **C99 back end:** collections, indexing and `switch` are not yet supported, and the `PRId64` portability defect should be fixed.
- **Check overhead:** static range analysis could remove checks that can never fail.
- **Interpreter speed:** a bytecode virtual machine would be considerably faster than tree walking.
- **Verification:** a formal semantics would allow the interpreter to be verified against it.

XIII. Conclusion

This project designed and implemented C−−, a small imperative language that obtains memory safety and freedom from undefined behaviour by removing dangerous features rather than adding protective machinery. The deliverables are a normative specification, a dependency-free toolchain with two execution back ends, beginner-friendly diagnostics, a CLI, an editor extension and a deployed web studio, validated by 445 tests on three operating systems. The evaluation shows that the most common weakness classes either cannot be written in C−− or stop deterministically with a clear explanation, at a measurable but moderate run-time cost in the compiled output. The source code, documentation and studio are openly available [20].

Acknowledgment

The authors thank Prof. Hariharan R L, Center for AI Research, School of Computer Science and Engineering (SCOPE), for his guidance and supervision throughout this project.

TABLE IX
WEAKNESS CLASSES AND HOW C−− REMOVES OR TRAPS THEM

Weakness (CWE)	C−− rule	Enforced by
NULL dereference (CWE-476)	No null value and no pointer types exist	grammar and type system
Buffer overflow via pointers (CWE-119, CWE-787)	No address-of, dereference or pointer arithmetic	grammar
Use of uninitialized variable (CWE-457)	Every declaration requires an initializer	parser (S001)
Use after free, aliasing bugs (CWE-416)	No references; collections are copied on assignment and call	semantics
Assignment in condition, ++/−− misuse	Assignment is a statement; `++`, `--`, `&`, `|` rejected	parser, lexer (L009–L012)
Implicit conversion, truthiness (CWE-704)	Conditions must be `bool`; assignment never converts	analyzer (E004, E007)
Accidental mutation	`let` bindings and parameters are deeply immutable	analyzer (E003, E028, E029)
Switch fall-through (CWE-484)	No fall-through; duplicate cases rejected	semantics, analyzer (E023)
Out-of-bounds index (CWE-125, CWE-787)	Every index is checked	runtime trap (R006)
Integer overflow (CWE-190)	All `int` operations checked in 64 bits	runtime trap (R001)
Division by zero (CWE-369)	Checked for `/` and `%` on both types	runtime trap (R002, R003)
Uncontrolled recursion (CWE-674)	Depth limited to 1000 calls on every platform	runtime trap (R007)

TABLE X
PERFORMANCE: INTERPRETER VS. TRANSPILED C99, AND CHECKED C99 VS. UNCHECKED HAND-WRITTEN C

	Experiment 1: execution back ends				Experiment 2: cost of checks			
Benchmark	N	**Interp. (s)**	**C99 (ms)**	**Speedup**	N	**Checked C99 (s)**	**Plain C (s)**	**Overhead**
Recursive Fibonacci	22	0.37	1.6	225×	35	0.046	0.018	+152%
Prime counting (trial division)	20 000	1.89	3.4	554×	2 000 000	0.989	0.566	+75%
Collatz step totals	3 000	1.50	1.7	878×	1 000 000	0.267	0.206	+29%
Leibniz series for π (float)	200 000	1.48	2.1	712×	200 000 000	0.675	0.213	+217%

REFERENCES

[1] M. Miller, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape," Presented at BlueHat IL, Tel Aviv, Israel, 2019.

[2] The Chromium Projects, "Memory safety," https://www.chromium.org/Home/chromium-security/memory-safety/, accessed: Oct. 2026.

[3] Cybersecurity and Infrastructure Security Agency (CISA) et al., "The case for memory safe roadmaps," CISA, NSA, FBI, and international partners, Tech. Rep., Dec. 2023.

[4] N. D. Matsakis and F. S. K. II, "The Rust language," *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014.

[5] T. Jim, J. G. Morrisett, D. Grossman, M. W. Hicks, J. Cheney, and Y. Wang, "Cyclone: A safe dialect of C," in *Proc. USENIX Annual Technical Conference*, 2002, pp. 275–288.

[6] G. C. Necula, S. McPeak, and W. Weimer, "CCured: Type-safe retrofitting of legacy code," in *Proc. 29th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, 2002, pp. 128–139.

[7] A. S. Elliott, A. Ruef, M. Hicks, and D. Tarditi, "Checked C: Making C safe by extension," in *Proc. IEEE Cybersecurity Development (SecDev)*, 2018, pp. 53–60.

[8] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "Address-Sanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conference*, 2012, pp. 309–318.

[9] MISRA, *MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems*. MIRA Ltd., 2013.

[10] J. Barnes, *High Integrity Software: The SPARK Approach to Safety and Security*. Addison-Wesley, 2003.

[11] B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, and J. Prather, "Compiler error messages considered unhelpful: The landscape of text-based programming error message research," in *Proc. Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR)*, 2019, pp. 177–210.

[12] E. Czaplicki, "Compiler errors for humans," https://elm-lang.org/news/compiler-errors-for-humans, 2015.

[13] F. J. Damerau, "A technique for computer detection and correction of spelling errors," *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.

[14] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[15] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[16] S. P. Jones, N. Ramsey, and F. Reig, "C--: A portable assembly language that supports garbage collection," in *Proc. Int. Conf. Principles and Practice of Declarative Programming (PPDP)*, ser. LNCS, vol. 1702, 1999, pp. 1–28.

[17] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std. 754-2019, 2019.

[18] ISO/IEC, "Programming languages — C," ISO/IEC 9899:1999, 1999.

[19] J. R. Allen, K. Kennedy, C. Porterfield, and J. Warren, "Conversion of control dependence to data dependence," in *Proc. 10th ACM SIGACT-SIGPLAN Symp. Principles of Programming Languages (POPL)*, 1983, pp. 177–189.

[20] R. Mexson, M. K. R. Panchal, and G. Gupta, "C-- (C Minus Minus): Language, Compiler, and Web Studio," https://github.com/ronitmexson2924/C--, 2026.